단계 3. 구조화 자료 매핑 how 보완

벤치마킹한 문헌의 원리와 우리 구현 과정

2026.10.30 · 문경민

1. 이번에 바뀐 점

핵심

- 문헌 표기를 바로잡았다. OntoDSMS 는 Zhou 외(2023)가 만든 댐 안전 모니터링 온톨로지의 이름이고, 관계형 DB 를 온톨로지에 연결한 기술은 기존 오픈소스 플랫폼 D2RQ 이다. 우리가 벤치마킹한 것은 D2RQ 의 매핑 원리 4 개이다.
- how 를 채웠다. 승인 표의 504 행 하나를 따라가며 식별자 → Class → 속성 → 근거 → 출처 순서로, 각 과정이 무엇을 어떻게 만들고 결과가 무엇인지 보였다.
- 결과를 보이게 했다. 결과 표 3 개의 한 줄이 무엇인지와 그것을 모은 Entity 의 모습(그림 5), 단계 3 이 끝났을 때 충주댐의 Entity 상태(그림 8)를 그렸다.
- 매핑을 검토하며 찾은 온톨로지의 부족한 점과 10/19 질문(AP 와 APR)을 검토할 점으로 정리했다.

표 1. 10/16 자료와 비교

항목	10/16 자료	이번
문헌 표기	"OntoDSMS(Zhou 2023)는 … 관계형 DB 의 자료를 온톨로지에 연결한다" — OntoDSMS 를 기술 이름처럼 씀	논문(Zhou 외 2023) · 온톨로지(OntoDSMS) · 연결 도구(D2RQ)를 구분. 벤치마킹 대상 = D2RQ 의 매핑 원리
문헌의 방법	표 → Class, 열 → 속성, 행 → 개체 (대응 원칙만)	원리 4 개와 각 원리의 예: 키 값으로 식별자, 표 → Class, 열 → 속성, 대응은 사람이 결정
우리 구현	Class 별 결과 예시 표	한 행을 따라가는 과정 5 개와 과정별 결과
결과	예시 값만	결과물이 CSV 파일 3 개임을 밝히고 파일 안의 실제 줄을 보임, 그것을 모은 Entity(그림 5), 단계 3 이 끝난 상태(그림 8)
검토할 점	없음	approvalContent 에 세 열이 합쳐짐, ID 접두어 표기
10/19 질문	—	매핑이 어떻게 작동하나 → 3 장 · 댐 기본 자료 → 3.3 · AP 와 APR → 과정 4, 4 장

2. 벤치마킹한 문헌의 원리: D2RQ 매핑

표기

- Zhou 외(2023): 댐 안전 모니터링을 위한 BIM·온톨로지 기반 지식관리를 다룬 논문
- OntoDSMS: 이 논문이 구축한 댐 안전 모니터링 시스템(DSMS) 도메인 온톨로지의 이름
- D2RQ: 이 논문이 만든 것이 아니라 기존 오픈소스 플랫폼. 논문은 MySQL 에 저장된 모니터링 데이터를 OntoDSMS 에 연결할 때 D2RQ 를 사용했다
- 본 연구가 벤치마킹한 것: D2RQ 의 매핑 원리
- 용어: 문헌의 instance 는 우리 용어로 Entity, DatatypeProperty 는 Data Property 이다. 이하 우리 용어로 쓴다

원리 1. 행의 키 값으로 Entity 의 식별자를 만든다

- D2RQ 는 식별자의 틀을 uriPattern 으로 정한다. 예: "item/@@item.item_ID@@"
- @@ 사이에 적힌 열(item_ID)의 값을 행마다 끼워 넣는다. item_ID 가 7 인 행 → item/7
- 식별자를 원본의 키 값으로 만들므로, 같은 행은 언제나 같은 식별자를 받는다

원리 2. 한 표에서 나온 식별자는 모두 같은 Class 의 Entity 가 된다

- D2RQ 는 d2rq:class :Item 으로, 그 표의 모든 행 식별자를 Item Class 의 Entity 로 선언한다
- 논문의 설명: "Converting a table in a database to a class in an ontology"

원리 3. 열 하나를 Data Property 하나에 대응시키고, 셀 값을 그 값으로 옮긴다

- D2RQ 는 열마다 어느 Entity 에 붙일지, 어떤 속성인지, 어느 열의 값을 옮길지를 선언한다. 예: column "item.Genre" → 속성 :itemName
- 고정 문자열과 열 값을 조합해야 할 때는 column 대신 pattern(틀)을 쓴다
- 논문의 설명: "Converting a column in a table to a DatatypeProperty in an ontology"

원리 4. 열과 속성의 대응은 사람이 온톨로지 용어로 정한다

- 매핑 파일은 DB 구조로부터 자동 생성되며, 이때 이름은 DB 의 표·열 이름 그대로이다
- 이를 사람이 온톨로지 용어에 맞게 고친다. 예: Genre 열 → :itemName 속성

3. 우리 구현: 승인 표 한 행이 Entity 가 되는 과정

- 단계 3 은 정형 표(승인 기록, K-water 측정자료)의 값을 해석을 더하지 않고 Ontology 의 Entity 와 Data Property 로 옮기는 단계이다.
- 아래는 승인 표의 504 행 하나를 따라간다. 그림의 노란 점선 상자는 그 과정에 쓰인 D2RQ 원리이다.

3.1 입력: 승인 표와 열의 역할 (원리 4)

- 입력: 전처리를 마친 승인 표(approval_records_clean.csv)에서 대상 4 개 댐(충주, 소양강, 횡성, 광동)의 80 행. 한 행은 승인 한 건이다.
- 변환하기 전에 열마다 어떤 역할을 할지 사람이 먼저 정했다. 표 2 의 머리 색이 역할이다.

표 2. 승인 표(일부)와 열의 역할

순차번호	관측소코드	관측소명	승인년월일시분	방류시작시간	접수방류량	비고
473	1006110	횡성	2011-06-24	2011-06-24 18:00	100	수문방류 승인
504	**1003110**	**충주**	**2011-06-27**	**2011-06-27 18:00**	**2000**	**(실제방류는 20 시부터)**
574	1003110	충주	2011-07-01	2011-07-01 18:00	2000	(실제 방류는 19 시부터)
식별자 키 (과정 1)	옮기지 않음 (댐 코드와 같은 값)	옮기지 않음 (코드와 같은 정보)	속성 하나 (과정 3)	세 열을 내용 속성 하나로 (과정 3)		

- 관측소코드: 자료 준비 때 승인 파일의 관측소코드와 K-water 의 댐 코드(DAM_CD)를 대조했고, 4 개 댐 모두 같은 코드였다(1003110 충주 ↔ 1003110 충주댐 등). 댐 자체의 정보는 댐 자료에서 만들므로(3.3) 승인 Entity 의 속성으로 옮기지 않는다.
- 관측소명: 관측소코드와 같은 정보라 옮기지 않는다.
- 이 밖에 원본 CSV 의 행 번호와 출처 열(기관, 내려받은 주소, 원본 파일명 등)이 있으며, 과정 4·5 에 쓴다.

3.2 과정 1~5

과정 1. 식별자 만들기 (원리 1)

- 틀 "APR:" + {순차번호}에 504 를 넣어 승인 Entity APR:504 를 만든다.
- 같은 행에서 틀 "EVI:AP:" + {순차번호}로 그 승인이 적힌 자료의 Entity EVI:AP:504 를 만든다.
- 순차번호를 키로 쓴 이유: 원본 자료가 승인 한 건마다 매긴 번호이고, 전처리에서 중복 행을 지운 뒤에는 표 안에서 유일하다. 그래서 80 행은 서로 다른 ID 80 쌍이 된다.
- ID 는 처리 순서나 무작위 번호가 아니라 원본의 순차번호로 만든다. 그래서 같은 행은 언제 다시 만들어도 같은 ID 가 된다.



순차번호	승인 Entity ID	근거 Entity ID
473	APR:473	EVI:AP:473
504	APR:504	EVI:AP:504
574	APR:574	EVI:AP:574
⋯ 80행 → 서로 다른 ID 80쌍		

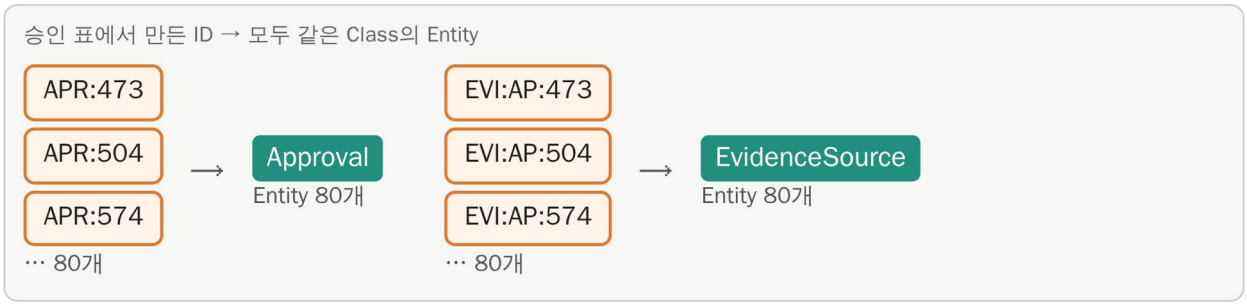
· 행 하나 → 승인 ID 하나 + 근거 ID 하나
· ID는 원본의 순차번호로 만든다
· 그래서 같은 행은 언제 다시 만들어도 같은 ID

원리 1 · D2RQ 구문 `d2rq:uriPattern "item/@@item.item_ID@@"`
@@ 사이 열(item_ID)의 값을 행마다 끼워 식별자를 만든다: item_ID = 7 → `item/7`

그림 1. 식별자 만들기 — 키 열의 값을 틀에 끼운다

과정 2. Class 지정 (원리 2)

- 승인 표에서 만든 APR: ID 80 개는 모두 Approval Class 의 Entity 로 기록한다.
- EVI:AP: ID 80 개는 모두 EvidenceSource Class 의 Entity 로 기록한다.



원리 2 · D2RQ `d2rq:class :Item` — 그 표의 모든 행 식별자를 Item Class의 Entity로 선언한다

그림 2. Class 지정

결과: CSV 파일 kgv4_entities.csv 에 줄이 생긴다 (한 줄 = Entity 하나)

Entity ID	Class
APR:504	Approval
EVI:AP:504	EvidenceSource

과정 3. 열 → Data Property (원리 3)

- 승인년월일시분 → approvalTime. 원본 값에 시각이 없어 앞 10 글자를 날짜로 저장한다(column).
- 방류시작시간, 접수방류량, 비고 → approvalContent 하나. 세 값을 글자 틀에 넣어 저장한다(pattern). 비고는 원문 그대로 넣는다.

- 세 열을 합친 이유: 주어진 온톨로지에 Approval 의 내용 속성이 approvalContent 하나뿐이었다(4 장 검토할 점 1).

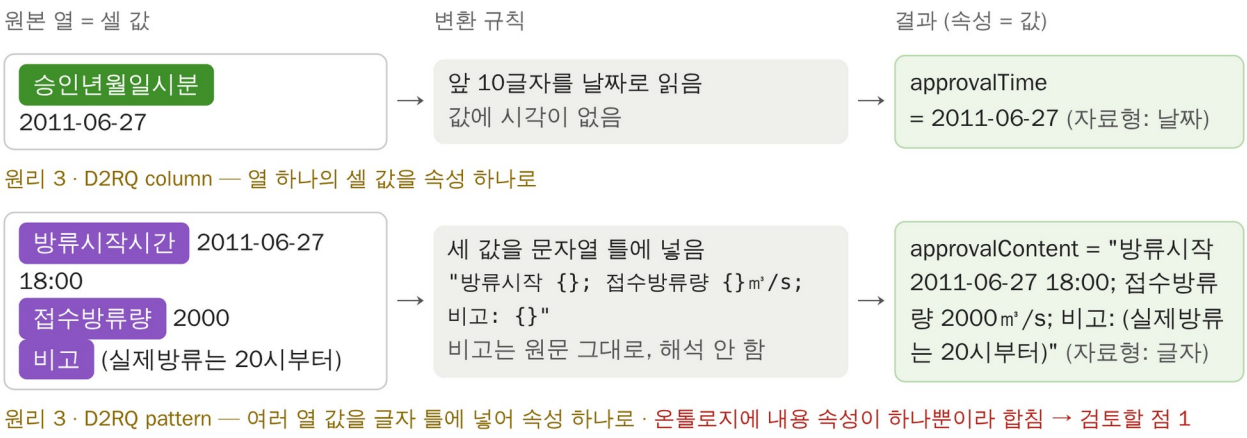


그림 3. 열 → Data Property — 원본 셀 값, 변환 규칙, 결과

결과: CSV 파일 kgv4_properties.csv 에 줄이 생긴다 (한 줄 = Entity 의 속성 하나)

Entity ID	속성	값	자료형
APR:504	approvalTime	2011-06-27	날짜
APR:504	approvalContent	방류시작 2011-06-27 18:00; 접수방류량 2000㎥/s; 비고: (실제방류는 20 시부터)	글자

과정 4. 근거 Entity 와 연결

- 같은 행의 출처 열을 근거 Entity EVI:AP:504 의 속성으로 옮긴다(아래 표).
- APR:504 와 EVI:AP:504 는 같은 504 행을 처리하면서 동시에 만든다. 그래서 만드는 그 자리에서 APR:504 —supportedBy→ EVI:AP:504 로 잇는다(그림 4).
- 두 ID 의 504 는 둘이 같은 원본 행에서 나왔다는 표시이다. 나중에 ID 글자(AP, APR)를 맞춰 보고 찾아서 잇는 것이 아니다.



그림 4. 같은 행에서 함께 만들며 잇는다 (APR:504 → EVI:AP:504)

결과: 근거 Entity 의 속성도 같은 CSV 파일 kgv4_properties.csv 에 줄로 추가된다

Entity ID	속성	값	자료형
EVI:AP:504	sourceTitle	환경부 한강홍수통제소 홍수예보 댐방류승인	글자
EVI:AP:504	sourceType	승인자료	글자
EVI:AP:504	sourceLocator	순차번호 504	글자

과정 5. 출처 기록

- 우리 원본은 DB 가 아니라 내려받은 CSV·JSON 파일이어서, 매핑한 값을 옮겨 저장한다.
- 값을 옮기면 KG 의 값과 원본 행의 연결이 끊긴다. 그래서 Entity 마다 원본 파일, 그 안의 위치, 적용한 규칙, 값의 성격을 출처 파일에 남긴다.
- 위치(504 번째 행)로 원 파일의 해당 행을 바로 열 수 있다. 값의 성격 "원본 그대로"는 원본 값을 가공 없이 옮겼다는 뜻이다.

결과: CSV 파일 kgv4_provenance.csv 에 줄이 생긴다 (한 줄 = Entity 의 출처 하나)

Entity ID	원본 파일	위치	규칙	값의 성격	메모
APR:504	approval_records_clean.csv	504 번째 행	MR-APR	원본 그대로	승인일은 날짜만 있음

결과: CSV 파일 3 개

- 단계 3 의 결과물은 CSV 파일 3 개이며, 03_normalized 폴더에 저장된다.
- kgv4_entities.csv: Entity ID 와 Class (한 줄 = Entity 하나)
- kgv4_properties.csv: Entity ID, 속성, 값, 자료형 (한 줄 = Entity 의 속성 하나)
- kgv4_provenance.csv: Entity ID, 원본 파일, 위치, 규칙, 값의 성격 (한 줄 = Entity 의 출처 하나)
- 세 파일의 줄을 Entity ID(APR:504)로 모으면 Entity 하나의 전체 모습이 된다: Class, 속성값, 출처(그림 5).

파일 안의 실제 모습 (첫 줄은 열 이름, 이어서 APR:504 의 줄)

```
kgv4_entities.csv
  entity_id,class,label
  APR:504,Approval,승인 504 (충주댐)

kgv4_properties.csv
  entity_id,property,value,datatype
  APR:504,approvalTime,2011-06-27,date
  APR:504,approvalContent,방류시작 2011-06-27 18:00; 접수방류량 2000㎥/s; 비고: (실제방류는 20
시부터),string

kgv4_provenance.csv
  item,kind,source_file,source_locator,rule,data_origin,note
  APR:504,entity,03_normalized/approval_records_clean.csv,row_number_in_file=504,MR-APR,real,
승인일은 날짜만 있음
```

kgv4_entities.csv 한 줄 = Entity 하나 (일부 열)

Entity ID	Class
APR:504	Approval

kgv4_properties.csv 한 줄 = Entity의 속성 하나

Entity ID	속성	값	자료형
APR:504	approvalTime	2011-06-27	날짜
APR:504	approvalContent	방류시작 2011-06-27 18:00; 접수방류량 2000㎥/s; 비고: (실제방류는 20시부터)	글자

kgv4_provenance.csv 한 줄 = Entity의 출처 하나

Entity ID	원본 파일	위치	규칙	값의 성격
APR:504	approval_records_clean.csv	504번째 행	MR-APR	원본 그대로

→

CSV 3개의 줄을 Entity ID로 모은 모습

APR:504 Approval

approvalTime = 2011-06-27 (날짜)

approvalContent = "방류시작 2011-06-27 18:00; 접수방류량 2000㎥/s; 비고: (실제방류는 20시부터)" (글자)

출처: approval_records_clean.csv 504번째 행 · 규칙 MR-APR · 원본 값 그대로

그림 5. CSV 파일 3 개의 줄과 그것을 모은 Entity (APR:504)

3.3 댐 자료와 측정 자료

Dam (댐 기본 자료)

- 입력: 댐 한 개가 한 행인 표는 따로 없다. K-water 일자료에는 댐 하나의 하루 측정값이 한 행이고, 행마다 댐 코드(DAM_CD), 댐 이름, 댐 유형이 반복해서 붙어 있다.
- 식별자: 틀 "DAM:" + {DAM_CD}로 만든다. 충주댐의 12,582 행은 모두 DAM_CD 가 1003110 이므로 모두 DAM:1003110 이 되고, Entity 하나로 모인다(원리 1).
- Class: Dam
- 속성: 댐 유형 열 → damType(다목적댐). damName 에는 승인 자료의 "충주"와 맞춘 표준 댐명(충주댐)을 넣는다.
- 결과: 승인과 같은 CSV 파일 3 개에 Dam Entity 의 줄이 추가된다.

K-water 일자료 (한 행 = 댐 하나의 하루 측정값 하나)

DAM_CD	댐 이름	댐 유형	날짜	변수	값
1003110	충주댐	다목적댐	2021-12-31	저수위	131.53
1003110	충주댐	다목적댐	2021-12-30	저수위	131.63
1003110	충주댐	다목적댐	2021-12-29	저수위	131.71
⋯ DAM_CD 1003110인 행 12,582개					

DAM_CD = 1003110 (12,582행 모두) → `"DAM:" + {DAM_CD}` → DAM:1003110 ID 하나 → Entity 하나

DAM:1003110 Dam

damName = 충주댐 (승인 자료의 "충주"와 맞춘 표준 댐명)

damType = 다목적댐 (댐 유형 열의 값)

원리 1 적용 — ID를 키 값(DAM_CD)으로 만들기 때문에, 같은 댐의 여러 행은 모두 같은 ID가 되어 Entity 하나로 모인다

그림 6. K-water 일자료 → Dam Entity

HydrometeorologicalState (HMS, 수문·기상 상태)

- 입력: K-water 시간자료. 한 행이 (댐 코드, 시각)이고 변수 6 개(수위, 저수량, 강우량, 유입량, 총방류량, 저수율)가 열로 펼쳐진 넓은 형태이며 553,609 행이다.
- 행마다 Entity 를 만들면 55 만 개가 된다. KG 에 필요한 것은 "어느 댐에 어떤 측정 변수가 있는가"이므로, 키 열(DAM_CD)과 열 머리(변수명)를 함께 틀에 넣어 (댐, 변수)마다 Entity 를 만든다. 예: HMS:1003110:수위
- 결과: 댐 4 × 변수 6 = 24 개. 속성은 variableType(수위 등)과 stationCode(1003110)이다.
- 시각별 값은 KG 에 넣지 않고 측정 저장소에 그대로 둔다.
- 같은 자료에서 댐마다 측정자료 근거 Entity EVI:KW:{DAM_CD}를 만들고, HMS 와 supportedBy 로 잇는다.
- 결과: 같은 CSV 파일 3 개에 HMS 와 근거 Entity 의 줄이 추가된다.

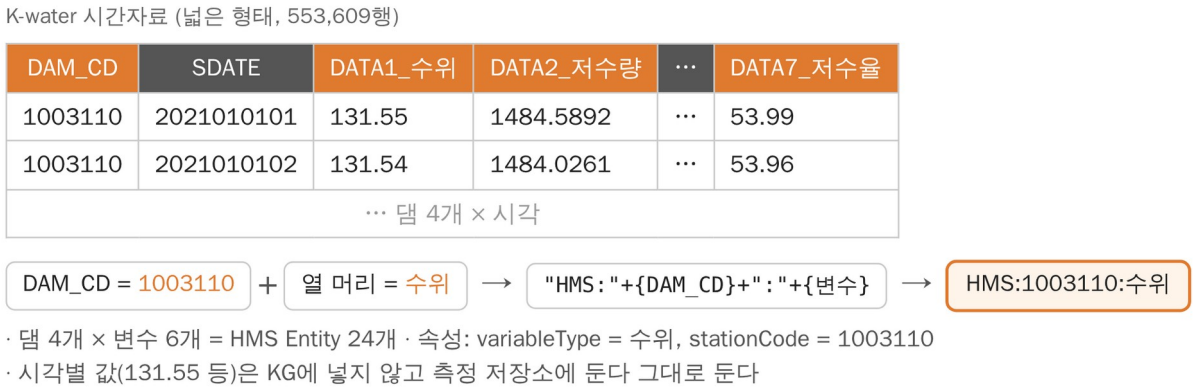
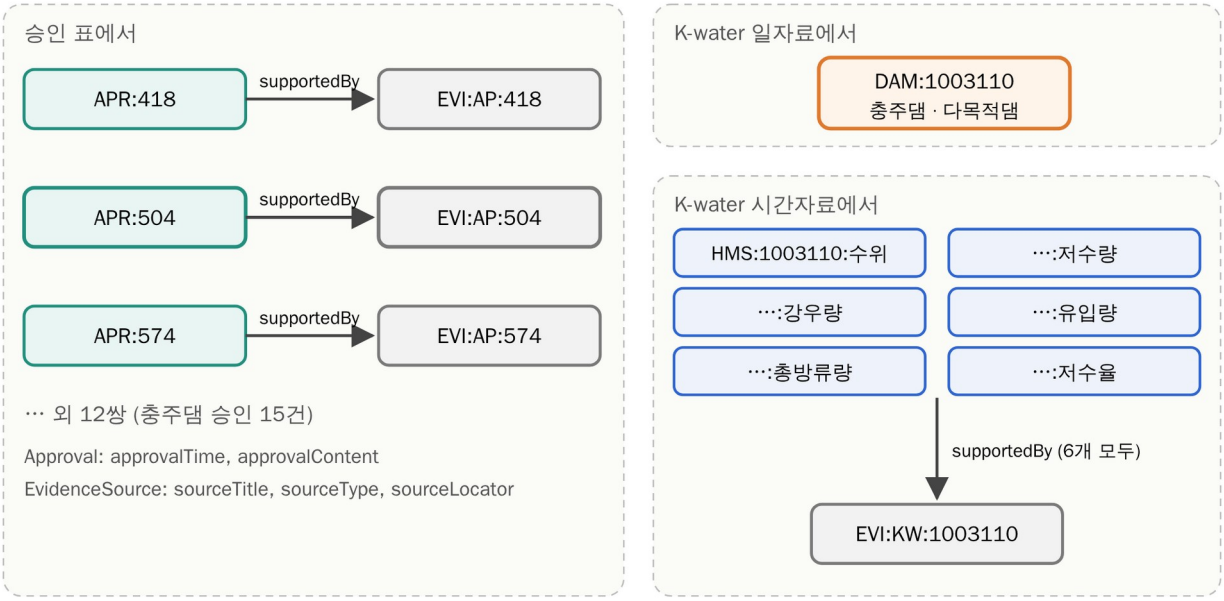


그림 7. K-water 시간자료 → HMS Entity

3.4 단계 3 이 끝난 상태

- 그림 8 은 충주댐에 관련된 Entity 를 만든 자료별로 모은 것이다.
- 각 자료에서 Entity 와 속성이 만들어졌고, 같은 행·같은 자료에서 함께 만든 근거만 supportedBy 로 이어져 있다.
- 서로 다른 자료에서 만든 승인, 댐, 측정 상태 Entity 는 아직 서로 떨어져 있다.
- 4 개 댐 전체: Approval 80, EvidenceSource 84(승인자료 80, 측정자료 4), HMS 24, Dam 4. 모두 Entity 192 개, 속성 468 개.



각 자료에서 Entity와 속성이 만들어졌고, 같은 행·같은 자료에서 함께 만든 근거만 supportedBy로 이어져 있다(실선). 서로 다른 자료에서 만든 승인, 댐, 측정 상태 Entity는 아직 서로 떨어져 있다.

그림 8. 단계 3 이 끝난 상태 — 충주댐의 Entity

4. 검토할 점 (온톨로지 발전 후보)

1. approvalContent 하나에 세 열이 들어가 있음

- 현재: 방류시작시간, 접수방류량, 비고를 글자 하나로 저장한다. 접수일자 열은 어느 속성에도 들어가지 않았다.
- 이유: 구축 당시 주어진 온톨로지에서 Approval 의 Data Property 는 approvalTime 과 approvalContent 두 개뿐이었고, 온톨로지는 수정하지 않는다는 전제로 매핑했다.
- 원리로 보면: 세 열은 원본에서 이미 나뉘어 있고 형태(시각, 숫자, 문장)도 분명하므로, 원리 3 대로라면 속성도 따로여야 한다.
- 할 수 없는 것: "접수방류량 2000㎥/s 이상인 승인", "승인일과 방류 시작의 시간 차" 같은 조건 질의
- 선택지 ①: Approval 에 Data Property 추가 — dischargeStartTime(일시), requestedDischarge(숫자, ㎥/s), remark(글자), 필요하면 receiptDate(날짜)
- 선택지 ②: 현재대로 두고, 승인 내용은 답변에 원문으로 보여 주는 용도로만 쓴다
- 판단 기준: 지금의 CQ4(과거 방류 승인)는 댐과 기간으로 승인을 찾아 내용을 그대로 돌려주므로 분리가 꼭 필요하지 않다. 방류량이나 시각을 조건으로 쓰는 질문을 CQ 에 넣을 때 함께 정한다.

2. ID 접두어 표기 (10/19 질문: AP 와 APR)

- 연결: APR:504 와 EVI:AP:504 는 같은 504 행에서 동시에 만들어지고, 그 자리에서 이어진다(과정 4, 그림 4). 그래서 글자가 달라도 연결에는 문제가 없다.
- 표기: APR 은 승인(Approval), EVI 는 근거 자료(EvidenceSource)이고, EVI 뒤의 AP 는 출처 종류(승인자료)이다. 측정자료는 KW, 규정은 LAW 이다.
- 검토: 사람이 볼 때 AP 와 APR 이 비슷해 헷갈린다. 출처 종류 표기를 Class 접두어와 맞출지(EVI:APR:504) 정한다.

부록. 문헌

- Zhou, Z. 외 (2023). BIM and ontology-based knowledge management for dam safety monitoring. Automation in Construction, 145, 104649.
- D2RQ 매핑에 대한 설명은 이 논문의 4 장, 매핑 파일의 예는 그림 3(수정된 매핑 파일 일부)에 있다.